

NOTAS DE CLASE



**FUNDAMENTACIÓN DE LA
ESPECIFICACIÓN DEL
PROBLEMA, REQUERIMIENTO
ESENCIAL PARA EL
DESARROLLO DE SISTEMAS
INFORMÁTICOS**

Edna Margarita Esteban Regino
Universidad Cooperativa de Colombia
Sede Montería

Documentos de docencia | Course Work
coursework.ucc.edu.co
N.º 35, noviembre de 2016
doi: <https://doi.org/10.16925/greylit.2050>

NOTA LEGAL

El presente documento de trabajo ha sido incluido dentro de nuestro repositorio de literatura gris por solicitud del autor, con fines informativos, educativos o académicos. Asimismo, los argumentos, datos y análisis incluidos en el texto son responsabilidad absoluta del autor y no representan la opinión del Fondo Editorial o de la Universidad.

DISCLAIMER

This coursework paper has been uploaded to our grey literature repository due to the request of the author. This document should be used for informational, educational or academic purposes only. Arguments, data and analysis included in this document represent authors' opinion not the Press or the University.



ACERCA DEL AUTOR

Edna Margarita Esteban Regino. Magíster en Tecnología Educativa y Medios Innovadores para la Educación, profesora instructora del programa de Ingeniería de Sistemas, Universidad Cooperativa de Colombia, sede Montería, Colombia.

Correo electrónico: edna.estebanr@campusucc.edu.co

CÓMO CITAR ESTE DOCUMENTO

E. M. Esteban Regino. *Fundamentación de la especificación del problema, requerimiento esencial para el desarrollo de sistemas informáticos*. (Documento de docencia N° 35). Bogotá: Ediciones Universidad Cooperativa de Colombia, 2016. Doi: <https://doi.org/10.16925/greylit.2050>

Este documento puede ser consultado, descargado o reproducido desde nuestro repositorio de documentos de trabajo (<http://coursework.ucc.edu.co>) para uso de sus contenidos, bajo la licencia de Creative Commons Reconocimiento-No Comercial-Sin Obra Derivada 4.0 Internacional. <http://creativecommons.org/licenses/by-nc-nd/4.0/>



TABLA DE CONTENIDO

INTRODUCCIÓN	6
Objetivo	6
1. CONCEPTOS BÁSICOS DE LOS SISTEMAS DE INFORMACIÓN	7
1.1 Definición de sistema de información	7
1.2 Niveles de toma de decisiones	7
1.3 Actividades básicas de los sistemas de información	7
1.4 Clasificación de los sistemas de información	8
2. FASES PARA LA CONSTRUCCIÓN DE UN SISTEMA DE INFORMACIÓN	9
2.1 Identificación de los problemas, oportunidades y objetivos	9
2.2 Determinación de los requerimientos humanos de información	9
2.3 Análisis de las necesidades del sistema	10
2.4 Diseño del sistema recomendado	10
2.5 Desarrollo y documentación del software	10
2.6 Prueba y mantenimiento del sistema e implementación	10
2.7 Evaluación del sistema	10
3. ESTRATEGIA DIDÁCTICA “APRENDIZAJE BASADO EN CASOS”	11
3.1 Problemas, soluciones y programas	11
3.2 Especificación del problema (Paso 1. Definición del problema)	11
3.3 El modelo del mundo del problema	13
3.4 Los requerimientos no funcionales	16
4. ACTIVIDAD ESPECIFICACIÓN DEL PROBLEMA	17
5. ANEXO 1. EJEMPLO ESPECIFICACIÓN DEL PROBLEMA	18
6. ANEXO 2. FORMATO ESPECIFICACIÓN DEL PROBLEMA	21
REFERENCIAS	22

ÍNDICE DE TABLAS

Tabla 1. Saberes esenciales	6
Tabla 2. Niveles de toma de decisiones	7
Tabla 3. Actividades básicas de los sistemas de información	7
Tabla 4. Clasificación de los sistemas de información	8
Tabla 5. Definición del problema	11
Tabla 6. Documentar los requerimientos funcionales del problema	13
Tabla 7. Identificar las entidades	14
Tabla 8. Modelar las características	15

ÍNDICE DE FIGURAS

Figura 1. Sistema de empleados.	12
Figura 2. Modelado de la clase Empleado y la clase Fecha.	15
Figura 3. Relación entre las entidades Empleado y Fecha	16

35 FUNDAMENTACIÓN DE LA ESPECIFICACIÓN DEL PROBLEMA, REQUERIMIENTO ESENCIAL PARA EL DESARROLLO DE SISTEMAS INFORMÁTICOS

Edna Margarita Esteban Regino

RESUMEN

Ante la necesidad de desarrollar la macrocompetencia hay que desarrollar sistemas informáticos aplicando el enfoque sistémico para proponer alternativas de solución a los problemas de las organizaciones y, en el caso específico del curso de análisis de sistemas, se requiere analizar los requerimientos de los sistemas informáticos para diseñar software de acuerdo a las necesidades del usuario utilizando normas, estándares y metodologías vigentes. Así pues, como estrategia para desarrollar dichas competencias, se proponen estas notas de clase, las cuales le brindan al estudiante una herramienta que le permite especificar el problema y descomponerlo en tres aspectos fundamentales: requerimiento funcional/requerimientos del usuario, el mundo o contexto, requerimientos no funcionales/las restricciones y condiciones que plantea el cliente. Esto, por medio de un formato para dicho fin y con un ejemplo con todos los pasos a desarrollar.

Palabras clave: especificación del problema, requerimientos funcionales, requerimientos no funcionales, sistemas informáticos.



INTRODUCCIÓN

En estas notas de clase se dan a conocer los conceptos básicos y las fases de los sistemas de información como marco de referencia, con énfasis en la especificación del problema. Para esto se parte del primer nivel, dedicado al desarrollo de problemas, soluciones y programas, de la estrategia didáctica “Aprendizaje basado en casos”, desarrollada en el libro *Fundamentos de Programación*; un proyecto de actualización curricular de la Universidad de los Andes (Bogotá, Colombia) cuyo principal propósito es encontrar mejores formas de enseñar y aprender a resolver problemas usando como herramienta un lenguaje de programación de computadores.

Para el desarrollo de la estrategia, en estas notas de clase se identifican tres aspectos del problema: 1. Requerimientos funcionales (los requerimientos del usuario); 2. El modelo del mundo del problema (el mundo en el que debe resolverse; y 3. Requerimientos no funcionales (las restricciones y condiciones que plantea el cliente). Todo esto por medio del estudio del caso empleado. Sin embargo, se recomienda

realizar una revisión de más casos directamente en el libro *Fundamentos de Programación*, en el inciso 4, “Casos de Estudios”. A partir de allí se plantean actividades académicas enmarcadas en la metodología interdisciplinaria centrada en equipos de aprendizaje (MICEA), las cuales conlleven al desarrollo de saberes esenciales. Estos saberes se describen en la tabla 1, con sus respectivos indicadores de desempeño y evidencias.

TABLA 1. Saberes esenciales

Competencias	Indicadores de desempeño	Indicadores de evidencias
Analizar los requerimientos de sistemas de información.	Establece los requerimientos para la solución de un problema.	Informe de análisis de requerimientos. Casos de estudio.

Fuente: elaboración propia

OBJETIVO

Desarrollar en el estudiante las competencias necesarias que le permitan desarrollar la competencia “Analizar los requerimientos de sistemas de información”, y pueda, de esta forma, proponer alternativas de solución a los problemas de las organizaciones.



I. CONCEPTOS BÁSICOS DE LOS SISTEMAS DE INFORMACIÓN

1.1 DEFINICIÓN DE SISTEMA DE INFORMACIÓN

Según Unicauca [1], un sistema de información se puede definir como un equipo que recupera, procesa, almacena, analiza datos y difunde información. También se puede decir que es un conjunto de procedimientos, operaciones o funciones que tienen como objetivo:

- Proporcionar datos oportunos y exactos que permitan tomar decisiones acertadas

con el fin de apoyar las actividades de una empresa o negocio.

- Garantizar información exacta y confiable, así como su almacenamiento, de tal forma que esté disponible cuando se necesite.
- Servir como herramienta para que los gerentes realicen planeación y control y disminuir la incertidumbre en la toma de decisiones en sus empresas mediante el suministro de información en todos los niveles de la organización: nivel inferior, nivel medio y nivel superior.

1.2 NIVELES DE TOMA DE DECISIONES

TABLA 2. Niveles de toma de decisiones

Niveles	Toma de decisiones	Ejemplo
Nivel inferior (operativo)	Rutinarias en las que se requiere difundir con facilidad el estado y las actividades actuales dentro de las funciones básicas del negocio.	Cantidad de materias primas requeridas, reabastecimiento de inventario.
Nivel medio	Planificación y control de operaciones. Implica implementar políticas formuladas por la administración superior.	Revisión de resúmenes y análisis de datos históricos.
Nivel superior	Decisiones a largo plazo enfocadas en la planificación estratégica, la asignación de recursos y la formulación de políticas.	Busca detallar todas las operaciones de la compañía y sus departamentos

Fuente: elaboración propia

1.3 ACTIVIDADES BÁSICAS DE LOS SISTEMAS DE INFORMACIÓN

TABLA 3. Actividades básicas de los sistemas de información

Actividades básicas de los sistemas de información	Descripción	Ejemplo
Entrada de información	Proceso mediante el cual el sistema de información toma los datos que requiere para procesar la información. Pueden ser: • Manuales: proporcionadas directamente por el usuario. • Automáticas: datos o información tomada de otros sistemas o módulos (interfaces automáticas).	Unidades típicas: terminales, cintas magnéticas, códigos de barras, escáner, monitores sensibles al tacto, teclado y ratón, entre otras.
Almacenamiento de información	Actividades más importantes de una computadora; en esta el sistema puede recordar la información guardada en la sesión o proceso anterior, dentro de estructuras de información denominadas "archivos".	Unidades típicas: discos magnéticos o discos duros, discos flexibles o disquetes y los discos compactos (CD-ROM).
Procesamiento de información	Capacidad del sistema de información para efectuar cálculos con datos introducidos recientemente, o bien almacenados a través de secuencias de operaciones preestablecidas. Esto permite la transformación de datos en una fuente de información que se puede utilizar en la toma de decisiones.	Generar una proyección financiera a partir de los datos que contiene un estado de resultados o un balance general de un año base.
Salida de información	Capacidad de un sistema de información para sacar la información procesada, o bien datos de entrada al exterior; dicha salida puede constituir la entrada a otro sistema de información o interface automática de salida.	Generar pólizas contables de los movimientos procesados de los clientes. Por medio de un sistema de control de clientes tiene una interfaz automática de salida con el sistema de contabilidad.

Fuente: elaboración propia



1.4 CLASIFICACIÓN DE LOS SISTEMAS DE INFORMACIÓN

TABLA 4. Clasificación de los sistemas de información

Clasificación	Definición	Ejemplo
Estructura organizacional	Se clasifican a lo largo de líneas estructuradas de la organización.	Sistemas para divisiones, departamentos, unidades de operación e incluso para empleados individuales.
Área funcional	Para todas las tareas rutinarias o repetitivas que se desarrollan en la empresa y que son esenciales para la operación de la organización.	Sistema de información contable, sistema de información de comercialización, y sistema de información de recursos humanos.
Ayuda brindada	Apoyan a los gerentes en la toma de decisiones o a empleados administrativos al momento de entregar un informe.	Gráficas, tablas, etc.

Fuente: elaboración propia



2. FASES PARA LA CONSTRUCCIÓN DE UN SISTEMA DE INFORMACIÓN

Una de las metodologías con la que los analistas llevan a cabo el análisis y el diseño de los sistemas de información es el ciclo de vida del desarrollo de sistemas (SDLC), desarrollado por Kendall y Kendall [2]. Esta metodología se divide en siete fases para el análisis y el diseño.

2.1 IDENTIFICACIÓN DE LOS PROBLEMAS, OPORTUNIDADES Y OBJETIVOS

Esta etapa es imprescindible para el éxito del resto del proyecto, ya que consiste en el análisis de lo que está ocurriendo en la empresa para posteriormente señalar los problemas. Las oportunidades residen en las situaciones que se puedan mejorar mediante el uso de sistemas de información con el fin de que la empresa puede obtener una ventaja competitiva o establecer un estándar en la industria. Un componente importante de esta primera fase es la identificación de los objetivos de la empresa con el propósito de determinar si el sistema de información puede ayudarla a alcanzarlos, al enfrentar sus problemas u oportunidades. Esta fase consiste en entrevistar a los usuarios, sintetizar el conocimiento obtenido, estimar el alcance del proyecto y documentar los resultados. Como resultado se obtiene un informe de viabilidad que contiene la definición de un problema y sintetiza los objetivos.

2.2 DETERMINACIÓN DE LOS REQUERIMIENTOS HUMANOS DE INFORMACIÓN

Fase en la cual se determinan las necesidades de los usuarios involucrados y se comprende la forma como interactúan en el contexto

laboral con sus sistemas de información actuales, a través de métodos interactivos tales como entrevistas, muestreos e investigación de datos duros. Además de los cuestionarios y los métodos discretos, se observa el comportamiento de los encargados al tomar las decisiones y sus entornos de oficina, así como los métodos integrales (e. g. la creación de prototipos). También es necesario saber cómo funciona la empresa y tener información completa sobre personas, objetivos, datos y procedimientos involucrados. En esta fase se responden preguntas tales como:

- ¿Cuáles son las fortalezas y limitaciones físicas de los usuarios?
- ¿Qué es necesario hacer para que el sistema sea perceptible, legible y seguro?
- ¿Cómo puede diseñarse el nuevo sistema a fin de que sea fácil de usar, aprender y recordar?
- ¿Cómo puede el sistema ser agradable o incluso divertido de usar?
- ¿Cómo puede el sistema apoyar las tareas laborales individuales de un usuario y buscar nuevas formas de hacerlas más productivas?
- ¿Cómo puede el sistema ofrecer un mejor apoyo para las tareas individuales que se deben llevar a cabo?
- ¿Qué nuevas tareas habilita el nuevo sistema que los usuarios no podían realizar sin él?
- ¿Cómo se puede crear el sistema de manera que extienda las capacidades de un usuario más allá de lo provisto por el sistema anterior?
- ¿Cómo puede el analista crear un sistema gratificante para los trabajadores?



2.3 ANÁLISIS DE LAS NECESIDADES DEL SISTEMA

En esta fase se determinan los requerimientos con el uso de herramientas tales como los diagramas de flujo de datos (DFD), con el fin de graficar la entrada, los procesos y la salida de las funciones de la empresa o los diagramas de actividad o de secuencia que permiten mostrar la sucesión de los eventos que ilustran los sistemas de una manera estructurada y gráfica. A partir de los diagramas de flujo de datos, se desarrolla un diccionario de datos que enlista todos los elementos de datos utilizados en el sistema —con sus especificaciones—, y se realiza un análisis de las decisiones estructuradas, las cuales son aquellas para las que se pueden determinar condiciones, alternativas de condición, acciones y reglas de acción. En esta fase se prepara la propuesta en la que se sintetiza todo lo que se ha averiguado sobre los usuarios, la capacidad de uso y la utilidad de los sistemas actuales.

2.4 DISEÑO DEL SISTEMA RECOMENDADO

Se utiliza la información recolectada antes con el fin de realizar el diseño lógico del sistema de información y se diseñan los procedimientos que ayudarán a que los usuarios introduzcan los datos con precisión. Así, los datos que entren al sistema de información serán los correctos mediante el uso de las técnicas del buen diseño de formularios y páginas web o pantallas, que consisten en el diseño de la interfaz que conecta al usuario con el sistema, de manera que esta sea perceptible, legible, segura, atractiva y divertida de usar. También se incluye el diseño de bases de datos que almacenarán gran parte de los datos necesarios para que los encargados estén capacitados para tomar las decisiones en la organización. Se trabaja con los usuarios el diseño de una salida (bien sea en pantalla, o bien impresa) que cumpla con sus necesidades de información.

2.5 DESARROLLO Y DOCUMENTACIÓN DEL SOFTWARE

Consiste en el trabajo conjunto entre el analista con los programadores con el fin de desarrollar el software original requerido y el desarrollo de una documentación efectiva para el software incluyendo manuales de procedimientos, ayuda en línea, sitios web con preguntas frecuentes (FAQ) y archivos Léame (Read Me) a incluir en el nuevo software. Se les indica a los usuarios cómo deben usar el software y qué hacer en caso de problemas.

2.6 PRUEBA Y MANTENIMIENTO DEL SISTEMA E IMPLEMENTACIÓN

Es necesario probar el sistema de información antes de utilizarlo ya que es mucho menos costoso detectar los problemas antes de entregar el sistema a los usuarios. Se completan una serie de pruebas que señalan los problemas con datos de muestra, y después se utilizan datos reales del sistema actual. Los planes de prueba se crean en las primeras etapas del SDLC y se refinan a medida que el proyecto progresa. Aunque esta fase se lleva a cabo de manera rutinaria durante toda la vida del sistema de información, en ocasiones también a través del sitio web del distribuidor.

2.7 EVALUACIÓN DEL SISTEMA

En la última fase se implementa el sistema de información y se capacita a los usuarios para operar el sistema. Los distribuidores se encargan de una parte de la capacitación, pero la supervisión de la capacitación es responsabilidad del analista de sistemas, al tiempo que se planea una conversión del sistema antiguo al nuevo. Esto al convertir los archivos de los formatos anteriores a los nuevos, o crear una base de datos, instalar equipo y llevar el nuevo sistema a producción.



3. ESTRATEGIA DIDÁCTICA “APRENDIZAJE BASADO EN CASOS”

3.1 PROBLEMAS, SOLUCIONES Y PROGRAMAS

Según Villalobos [3], el primer factor que se debe tener en cuenta al resolver problemas y plantear soluciones desde un programa de computador es la motivación y, en el caso de problemas grandes (como el sistema de información de una empresa), complejos (como crear una visualización tridimensional de un diseño) o críticos (como controlar un tren), la solución la construyen equipos de ingenieros de software entrenados especialmente para asumir un reto de esa magnitud. Sin embargo, independientemente del tamaño del problema, surgen algunos interrogantes:

- ¿Cómo se define un problema?
- ¿Cómo, a partir del problema, se construye un programa para resolverlo?
- ¿De qué está conformado un programa?
- ¿Cómo se construyen sus partes?
- ¿Cómo hacer para que el computador entienda la solución?

A fin de resolver el problema, se siguen tres pasos: la definición del problema, el proceso de construcción de la solución y el contenido y la estructura de la solución. Para el caso de estas notas de clase, se desarrolla solo el primer paso: “la definición del problema”.

3.2 ESPECIFICACIÓN DEL PROBLEMA (PASO 1. DEFINICIÓN DEL PROBLEMA)

La base para determinar qué debe lograr un sistema de información consiste en definir el problema a partir de la identificación los

problemas. Se entiende por problemas los que se obtienen comprobando las salidas de los procesos organizacionales, al observar el comportamiento de los empleados y escuchar la retroalimentación externa de distribuidores, clientes y proveedores. La definición de un problema se sintetiza en uno o dos párrafos seguidos de una serie de cuestiones que describen la situación actual y una serie de objetivos o metas que son la situación deseada. Para definir el problema es necesario recopilar información que se obtiene a través de entrevistas, observaciones y analizar los documentos con los usuarios. Con la definición del problema se obtienen los requerimientos funcionales (cosas que se deben lograr o posibles soluciones), y los requerimientos no funcionales (restricciones que limitan el desarrollo del sistema) [2].

TABLA 5. Definición del problema

Identificadores	Señales específicas
Revisar las salidas de los procesos organizacionales y compararlas con los criterios de rendimiento.	• Demasiados errores. • El trabajo se completa con lentitud. • El trabajo se hace en forma incorrecta. • El trabajo se hace en forma incompleta. • No se realiza ningún trabajo.
Observar el comportamiento de los empleados	• Niveles altos de ausentismo. • Mucha inconformidad en el trabajo. • Mucha rotación de empleados.
Escuchar la retroalimentación externa de distribuidores, clientes y proveedores	• Quejas. • Sugerencias para mejorar. • Pérdida de ventas. • Ventas más bajas.

Fuente: tomado de [2]

Se parte del hecho de que no se puede resolver un problema que no se entiende. Por ello, la primera etapa en todo proceso de construcción de software consiste en tratar de entender el problema que tiene el cliente y expresarlo de manera tal que cualquier persona del equipo de desarrollo entienda sin dificultad lo que espera el cliente de la solución. Dicha etapa se denomina “análisis”, y la salida de esta etapa “especificación del problema”.

Lo primero que se debe hacer para tratar de entender y especificar el problema es identificar, por lo menos, tres de sus aspectos:



- Requerimiento funcional (los requerimientos del usuario o el servicio que el programa debe proveer al usuario).
- El mundo en el que debe resolverse el problema o contexto en el que ocurre el problema (estructura y funcionamiento de la empresa).
- Requerimientos no funcionales o las restricciones y condiciones que plantea el cliente, usados para limitar las soluciones posibles (tiempos de entrega, cantidad de usuarios).

3.2.1 Requerimientos funcionales

Definir los requerimientos funcionales es el primer componente del análisis. Estos se encuentran directamente relacionados con el problema que se quiere resolver, el cual se puede ver como un servicio que el programa le ofrece al usuario para resolver una parte del problema. A fin de determinar si algo es o no un requerimiento funcional, es conveniente plantearse tres preguntas:

- ¿Poder realizar esta operación es una de las razones por las cuales el cliente necesita construir un programa?
- ¿La operación no es ambigua? (la idea es descartar que haya más de una interpretación posible de la operación).
- ¿La operación tiene un comienzo y un fin? (buscar operaciones puntuales que correspondan a acciones que puedan ser hechas por el usuario).

Un requerimiento funcional se describe a través de cuatro elementos:

- Un identificador y un nombre.
- Un resumen de la operación.

- Las entradas (datos) que debe proporcionar el usuario para que el programa pueda realizar la operación.
- El resultado esperado de la operación. Hay tres tipos posibles de resultado: 1. Una modificación de un valor en el mundo del problema; 2. El cálculo de un valor; 3. Una mezcla de los dos anteriores.

3.2.1.1 Caso de Estudio N.º 1: Un empleado.

Se considera un programa que administra la información de un empleado. El empleado tiene un nombre, un apellido, un género (masculino o femenino), una fecha de nacimiento y una imagen asociada (su foto). Además, tiene una fecha de ingreso a la empresa en la que trabaja y un salario básico asignado. Desde el programa debe ser posible cambiar el salario del empleado, así como realizar los siguientes cálculos con la información disponible:

- Edad actual.
- Antigüedad en la empresa.
- Prestaciones a las que tiene derecho en la empresa (para el cálculo de las prestaciones se utiliza la fórmula: $P = (A * s)/12$ (P: prestaciones, A: antigüedad, s: salario).

FIGURA 1. Sistema de empleados.

Fuente: tomada de [4]



3.2.1.2 Documentar los requerimientos funcionales del problema.

TABLA 6. Documentar los requerimientos funcionales del problema

Nombre	R1: ingresar la información del empleado.	Se asocia el identificador R1 con el requerimiento. En la mayoría de los casos el identificador del requerimiento se asigna siguiendo alguna convención definida por la empresa de desarrollo. Se utiliza el verbo "ingresar" para describir la operación que se quiere realizar.
Resumen	Permite al usuario ingresar la información del empleado: datos personales y datos de vinculación a la empresa.	Se describe la operación y se proporciona una idea global del tipo de información que se debe ingresar y del resultado obtenido.
Entradas	Nombre del empleado, Apellido del empleado, Sexo del empleado, Fecha de nacimiento, Fecha de ingreso a la compañía, Salario básico.	En este caso se necesitan seis entradas para realizar el requerimiento, información que debe proveer el usuario al programa. En ella no se define la manera en que dicha información será ingresada por el usuario, puesto que eso va a depender del diseño que se haga de la interfaz y será una decisión que se tomará más tarde en el proceso de desarrollo. Tampoco se habla del formato en el que va a entrar la información. Por ahora, solo se necesita entender, de manera global, lo que el cliente quiere que el programa sea capaz de hacer.
Resultado	Nuevo salario	La operación corresponde de nuevo a una modificación de algún valor del mundo, puesto que con la información obtenida como entrada se quieren modificar los datos del empleado.
Nombre	R2: modificar el salario básico del empleado.	Es conveniente asociar un identificador con cada requerimiento, y así hacer fácilmente referencia a él. En este caso el identificador es R2. Es aconsejable que el nombre de los requerimientos corresponda a un verbo en infinitivo, para dar una idea clara de la acción asociada con la operación. En este ejemplo el verbo asociado con el requerimiento es "modificar".
Resumen	Permite modificar el salario básico del empleado.	El resumen es una frase corta que explica sin mayores detalles el requerimiento funcional.
Entradas	Nuevo salario	Las entradas corresponden a los valores que debe suministrar el usuario al programa para poder resolver el requerimiento. Un requerimiento puede tener cero o muchas entradas. Cada entrada debe tener un nombre que indique claramente su contenido. No es buena idea utilizar frases largas para definir una entrada.
Resultado	El salario del empleado ha sido actualizado con el nuevo salario.	El resultado del requerimiento funcional de este ejemplo es una modificación de un valor en el mundo del problema: el salario del empleado cambió. Un ejemplo de un requerimiento que calcula un valor podría ser aquél que informa la edad del empleado. Fíjese que el hecho de calcular esta información no implica la modificación de ningún valor del mundo del problema. Un ejemplo de un requerimiento que modifica y calcula a la vez, podría ser aquél que modifica el salario del empleado y calcula la nueva retención en la fuente.

Fuente: [4]

3.3 EL MODELO DEL MUNDO DEL PROBLEMA

En el segundo componente del análisis el objetivo es entender el mundo en el que ocurre el problema y recopilar toda la información necesaria para que el programador pueda escribirlo. En este componente no se analizan las opciones que el cliente quiere que tenga el programa, sino se debe entender cómo es el mundo en el que ocurre el problema. Esta actividad está basada en un proceso de "observación" del mundo del problema; puesto que los elementos que allí aparecen ya existen, el objetivo no es opinar sobre ellos (o proponer cambiarlos), sino simplemente identificarlos y describir la forma como se relacionan con el problema. Este modelo será utilizado en la etapa de diseño con el fin de definir los elementos básicos del programa.

Son cuatro las actividades que deben realizar el programador para construir el modelo del mundo:

- Identificar las entidades del mundo.
- Documentar las características de cada una de las entidades.
- Definir las relaciones que existen entre las distintas entidades.
- Documentar la información adicional (reglas, restricciones, etc.) que se tenga sobre las entidades.

Para expresar el modelo del mundo se utilizará la sintaxis definida en el diagrama de clases del lenguaje de modelos UML (Unified Modeling Language). Dicho lenguaje es un



estándar definido por la organización Object Management Group (OMG) y utilizado por una gran cantidad de empresas en el mundo para expresar sus modelos.

3.3.1 Identificar las entidades

Esta primera actividad tiene como objetivo identificar los elementos del mundo que intervienen en el problema y asociarles un nombre significativo. Dichos elementos pueden ser concretos (una persona, un vehículo), o abstractos (una cuenta bancaria). Una primera pista para localizarlos es buscar los sustantivos del enunciado del problema. Esto sirve en el caso de problemas pequeños, pero no es

generalizable a problemas de mayor dimensión. En programación orientada a objetos, las entidades del mundo se denominan “clases”, y serán los elementos básicos del diseño y la posterior implementación.

En el caso del empleado, hay dos entidades en el mundo del problema: la clase Empleado y la clase Fecha. Esta última se emplea para representar el concepto de fecha de nacimiento y fecha de ingreso a la empresa. Estos son los únicos elementos del mundo del problema que se mencionan. Lo demás corresponden a características de dichas entidades (el nombre, el apellido, etc.) o a requerimientos funcionales.

TABLA 7. Identificar las entidades

Entidad	Descripción
Empleado	Es la entidad más importante del mundo del problema. Resulta una buena práctica comenzar la etapa de análisis tratando de identificar la clase más importante del problema.
Fecha	Este es otro concepto que existe en el mundo del problema. Los nombres asignados a las clases deben ser significativos y dar una idea clara de la entidad del mundo que representan. Cuando el nombre de la entidad es compuesto, se usa por convención una letra mayúscula al comienzo de cada palabra. No se debe exagerar con la longitud del nombre, porque de lo contrario los programas pueden resultar pesados de leer. En el lenguaje Java no es posible usar tildes en los nombres de las clases.

Fuente: [4]

3.3.2 Modelar las características

Una vez se han identificado las entidades del mundo del problema, el siguiente paso es identificar y modelar sus características. A cada característica que ha encontrado se le debe asociar:

- Un nombre significativo.
- Una descripción del conjunto de valores que dicha característica puede tomar.

En programación orientada a objetos, las características se denominan “atributos” y, al igual que las clases, serán elementos fundamentales tanto en el diseño como en la implementación. El nombre de un atributo debe ser una cadena de caracteres no vacía que

empiece con una letra y no contenga espacios en blanco.

Por convención, el nombre de los atributos comienza por una letra minúscula. Si es un nombre compuesto se debe iniciar cada palabra simple con mayúscula.

En el lenguaje UML, una clase se dibuja como un cuadrado con tres zonas. La primera de ellas con el nombre de la clase y la segunda con sus atributos. El objetivo es mostrar la manera de identificar y modelar los atributos de una clase.

En este ejemplo se identifican las características de las clases Empleado y Fecha para el caso de estudio del empleado.

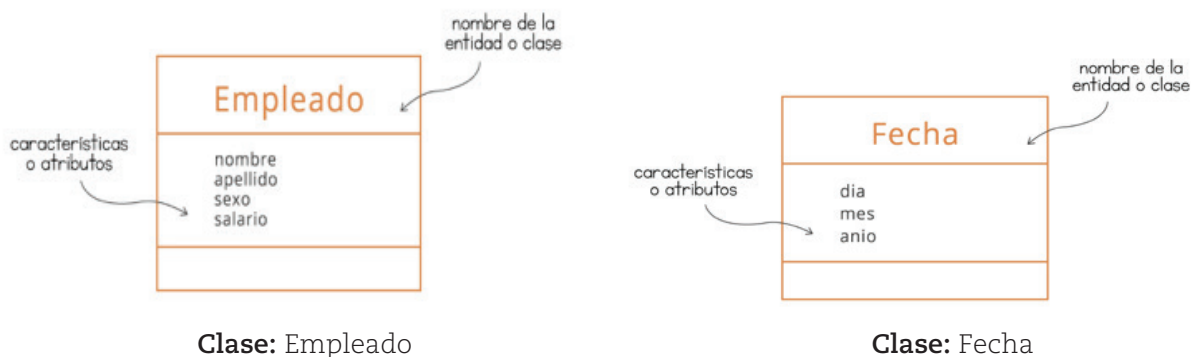


FIGURA 2. Modelado de la clase Empleado y la clase Fecha.

Fuente: [4]

TABLA 8. Modelar las características

Clase: Empleado		
Atributo	Valores Posibles	Comentarios
Nombre	Cadena de caracteres	La primera característica que aparece en el enunciado es el nombre del empleado. El valor de este atributo es una cadena de caracteres (por ejemplo, "Juan"). Se selecciona "nombre" como nombre del atributo. Es importante que los nombres de los atributos sean significativos (deben dar una idea clara de lo que una característica representa), para facilitar así la lectura y la escritura de los programas.
Apellido	Cadena de caracteres	El segundo atributo es el apellido del empleado. Al igual que en el caso anterior, el valor que puede tomar este atributo es una cadena de caracteres (por ejemplo, "Pérez"). Como nombre del atributo se selecciona "apellido". El nombre de un atributo debe ser único dentro de la clase (no es posible dar el mismo nombre a dos atributos).
Sexo	Masculino o femenino	Esta característica puede tomar dos valores: masculino o femenino. En esta etapa de análisis basta con identificar los valores posibles. Es importante destacar que los valores posibles de este atributo (llamado "sexo") no son cadenas de caracteres. No interesan las palabras en español que pueden describir los valores posibles de esta característica, sino los valores en sí mismos.
Salario	Valores enteros positivos	El salario está expresado en pesos y su valor es un número entero positivo. Aquí se está suponiendo que el valor del salario no tiene centavos.
Clase: Fecha		
Atributo	Valores posibles	Comentarios
Día	Valores enteros entre 1 y 31	La primera característica de una fecha es el día, y puede tomar valores enteros entre 1 y 31. En los nombres de las variables no puede haber tildes.
Mes	Valores enteros entre 1 y 12	La segunda característica es el mes. Aquí se podrían listar los meses del año como los valores posibles (por ejemplo, enero, febrero, etc.), pero por simplicidad se dirá que el mes corresponde a un valor entero entre 1 y 12.
Año-anio	Valores enteros positivos	La última característica es el año. Debe ser un valor entero positivo (por ejemplo, 2001). Aquí se encuentra de nuevo con un problema en español: los nombres de los atributos no pueden contener la letra "ñ". En este caso se resolvió reemplazar dicha letra y llamar el atributo "anio", que da aproximadamente el mismo sonido. Con las tres características anteriores queda completamente definida una fecha. Esa es la pregunta que se debe hacer en esta etapa: ¿es necesaria más información para describir la entidad que se está representando? Si se encuentra una característica cuyos valores posibles no son simples tales como números, cadenas de caracteres o una lista de valores, hay que preguntarse si dicha característica no es más bien otra entidad que no se identificó en la etapa anterior. Si es el caso, simplemente se debe agregar.

Fuente: Adaptado de [4]



3.3.3 Relaciones entre entidades

En esta actividad se identificarán las relaciones que existen entre las distintas entidades del mundo y se les asignará un nombre. Las relaciones se representan en UML como flechas que unen a las cajas de las clases y se denominan usualmente “asociaciones”. El diagrama de clases en el cual se incluye la representación de todas las entidades y las relaciones que existen entre ellas, se conoce como el “modelo conceptual”, porque explica la estructura y las relaciones de los elementos del mundo del problema.

El nombre de la asociación sigue las mismas convenciones del nombre de los atributos y debe reflejar la manera en que una clase

utiliza a la otra como parte de sus características. Es posible tener varias relaciones entre dos clases, y por eso es importante seleccionar bien el nombre de cada asociación. Una asociación se puede ver como una característica de una entidad cuyo valor está representado por otra clase.

Para el caso de estudio del Empleado, la primera asociación dice que un empleado tiene una fecha de nacimiento y que esta fecha es una entidad del mundo, representada por la clase Fecha. La segunda asociación hace lo mismo con la fecha de ingreso del empleado a la empresa. La dirección de la flecha indica la entidad que “contiene” a la otra. El empleado tiene una fecha, pero la fecha no tiene un empleado.



FIGURA 3. Relación entre las entidades Empleado y Fecha

Fuente: [4]

3.4 LOS REQUERIMIENTOS NO FUNCIONALES

En la mayoría de los casos, la solución que se va a construir debe tener en cuenta las restricciones definidas por el cliente que dependen, en gran medida, del contexto de utilización del programa. Para el caso del empleado, por ejemplo, el cliente podría pedir que el programa se pueda usar a través de un teléfono celular, o desde un navegador de Internet, o que el tiempo de respuesta de cualquier consulta sea menor a 0,0001 segundos.

Los requerimientos no funcionales están muchas veces relacionados con restricciones

sobre la tecnología que se debe usar, el volumen de los datos que se debe manejar o la cantidad de usuarios. Para problemas grandes, los requerimientos no funcionales son la base para el diseño del programa. Es distinto un programa que debe trabajar con un único usuario, de otro que debe funcionar con miles de ellos de forma simultánea.

Dados los objetivos y el tamaño de los problemas, solo se consideran los requerimientos no funcionales de interacción y visualización que están ligados con la interfaz de los programas.



4. ACTIVIDAD ESPECIFICACIÓN DEL PROBLEMA

INSTRUCCIÓN	
Nombre de la Actividad	Especificación del problema
Semana	Semana 16
Tiempo	4 horas
Estrategia MICEA	Actividad Individual
Procedimiento	1. Leer el libro <i>Análisis y Diseño de Sistemas</i> de Kendall y Kendall [2] Capítulo 2 2. Leer el libro <i>Análisis y Diseño de Sistemas</i> de Kendall y Kendall [2] Capítulo 7 3. Leer la Guía para especificación del problema 4. Revisar Ejemplo de especificación del problema 5. A partir del informe de requerimientos realizar las siguientes actividades: • {1} Dar nombre al sistema a desarrollar • {2} Hacer la descripción del problema • {3} Identificar los aspectos que hacen parte del problema • {4} Documentar los requerimientos funcionales • {5} Modelar el Mundo del problema • {6} Documentar requerimientos no funcionales • {7} Elegir una herramienta case y diseñar el Mockups del Sistema.
Producto a entregar	Informe Word, arial 12, Formato para entregar la actividad
Porcentaje	



5. ANEXO 1. EJEMPLO ESPECIFICACIÓN DEL PROBLEMA

Ejemplo del análisis y especificación del problema	
1. Nombre del sistema:	Sistema de empleados
2. Descripción del problema	
Se requiere una aplicación que permita manejar la información de un empleado. El empleado tiene un nombre, un apellido, un género (masculino o femenino), una fecha de nacimiento y una imagen asociada (su foto). Además, tiene una fecha de ingreso a la empresa en la que trabaja y un salario básico asignado. Desde la aplicación se puede: 1. Modificar el salario del empleado; y 2. Realizar algunos cálculos con la información disponible: (a) edad actual, (b) antigüedad en la empresa y (c) prestaciones a las que tiene derecho. Para el cálculo de las prestaciones se utiliza la fórmula $P = (A * s) / 12$ (P: prestaciones, A: antigüedad, s: salario).	

3. Identificar los aspectos que hacen parte de un problema	
Cliente	La empresa x
Usuario	Auxiliar talento humano
Requerimiento funcional	R1: Ingresar la información del empleado. Permite al usuario ingresar la información del empleado: datos personales y datos de vinculación a la empresa. R2: actualizar el salario básico del empleado. Permite modificar el salario básico del empleado.
Mundo del problema	El empleado tiene un nombre, un apellido, un género (masculino o femenino), una fecha de nacimiento y una imagen asociada (su foto). Además, tiene una fecha de ingreso a la empresa en la que trabaja y un salario básico asignado.
Requerimiento no funcional	Debe ser posible usar el programa a través de un teléfono celular, o desde un navegador de Internet, y que el tiempo de respuesta de cualquier consulta sea menor a 0,0001 segundos.

4. Documentación de los requerimientos funcionales del problema		
Nombre	R1: ingresar la información del empleado.	Asociamos el identificador R1 con el requerimiento. En la mayoría de los casos, el identificador del requerimiento se asigna siguiendo alguna convención definida por la empresa de desarrollo. Utilizamos el verbo "ingresar" para describir la operación que se quiere realizar.
Resumen	Permite al usuario ingresar la información del empleado: datos personales y datos de vinculación a la empresa.	Describimos la operación, y se proporciona una idea global del tipo de información que se debe ingresar y del resultado obtenido.
Entradas	Nombre del empleado, Apellido del empleado, Sexo del empleado, Fecha de nacimiento, Fecha de ingreso a la compañía, Salario básico.	En este caso se necesitan seis entradas para poder realizar el requerimiento. Información que debe proveer el usuario al programa. Note que no se define la manera en que dicha información será ingresada por el usuario, puesto que eso va a depender del diseño que se haga de la interfaz, y será una decisión que se tomará más tarde en el proceso de desarrollo. Fíjese que tampoco se habla del formato en el que va a entrar la información. Por ahora solo se necesita entender, de manera global, lo que el cliente quiere que el programa sea capaz de hacer.
Resultado	Nuevo salario	La operación corresponde de nuevo a una modificación de algún valor del mundo, puesto que con la información obtenida como entrada se quieren modificar los datos del empleado.
Nombre	R2: actualizar el salario básico del empleado.	Es conveniente asociar un identificador con cada requerimiento, para hacer fácilmente referencia a él. En este caso el identificador es R2. Es aconsejable que el nombre de los requerimientos corresponda a un verbo en infinitivo, para dar una idea clara de la acción asociada con la operación. En este ejemplo el verbo asociado con el requerimiento es "actualizar".
Resumen	Permite modificar el salario básico del empleado.	El resumen es una frase corta que explica sin mayores detalles el requerimiento funcional.
Entradas	Nuevo salario	Las entradas corresponden a los valores que debe suministrar el usuario al programa para resolver el requerimiento. En el requerimiento del ejemplo, si el usuario nos da como entrada el nuevo salario que quiere asignar al empleado, el programa no podrá hacer el cambio. Un requerimiento puede tener cero o muchas entradas. Cada entrada debe tener un nombre que indique claramente su contenido. No es buena idea utilizar frases largas para definir una entrada.
Resultado	El salario del empleado ha sido actualizado con el nuevo salario.	El resultado del requerimiento funcional de este ejemplo es una modificación de un valor en el mundo del problema: el salario del empleado cambió. Un ejemplo de un requerimiento que calcula un valor podría ser aquél que informa la edad del empleado. Fíjese que el hecho de calcular esta información no implica la modificación de ningún valor del mundo del problema. Un ejemplo de un requerimiento que modifica y calcula a la vez, podría ser aquél que modifica el salario del empleado y calcula la nueva retención en la fuente.



5. El modelo del mundo del problema

5.1. Identificar las entidades

Entidad	Descripción
Empleado	Es la entidad más importante del mundo del problema. Resulta una buena práctica comenzar la etapa de análisis tratando de identificar la clase más importante del problema.
Fecha	Este es otro concepto que existe en el mundo del problema. Los nombres asignados a las clases deben ser significativos y dar una idea clara de la entidad del mundo que representan. Cuando el nombre de la entidad es compuesto, se usa por convención una letra mayúscula al comienzo de cada palabra. No se debe exagerar con la longitud del nombre, porque de lo contrario, los programas pueden resultar pesados de leer. En el lenguaje Java no es posible usar tildes en los nombres de las clases.

5.2. Modelar las características

Nombre de la clase:		Empleado
Modelado de la clase:		
Atributo	Valores posibles	Comentarios
Nombre	Cadena de caracteres	La primera característica que aparece en el enunciado es el nombre del empleado. El valor de este atributo es una cadena de caracteres (por ejemplo, "Juan"). Seleccionamos "nombre" como nombre del atributo. Es importante que los nombres de los atributos sean significativos (deben dar una idea clara de lo que una característica representa), para facilitar así la lectura y la escritura de los programas.
Apellido	Cadena de caracteres	El segundo atributo es el apellido del empleado. Al igual que en el caso anterior, el valor que puede tomar este atributo es una cadena de caracteres (por ejemplo, "Pérez"). Como nombre del atributo seleccionamos "apellido". El nombre de un atributo debe ser único dentro de la clase (no es posible dar el mismo nombre a dos atributos).
Sexo	Masculino o femenino	Esta característica puede tomar dos valores: masculino o femenino. En esta etapa de análisis basta con identificar los valores posibles. Es importante destacar que los valores posibles de este atributo (llamado "sexo") no son cadenas de caracteres. No nos interesan las palabras en español que pueden describir los valores posibles de esta característica, sino los valores en sí mismos.
Salario	Valores enteros positivos	El salario está expresado en pesos y su valor es un número entero positivo. Aquí se está suponiendo que el valor del salario no tiene centavos.

Nombre de la clase:		Fecha
Modelado de la clase:		
Atributo	Valores Posibles	Comentarios
Dia	Valores enteros entre 1 y 31	La primera característica de una fecha es el día y puede tomar valores enteros entre 1 y 31. En los nombres de las variables no puede haber tildes.
Mes	Valores enteros entre 1 y 12	La segunda característica es el mes. Aquí se podrían listar los meses del año como los valores posibles (por ejemplo, enero, febrero, etc.), pero por simplicidad se dirá que el mes corresponde a un valor entero entre 1 y 12.
Anio	Valores enteros positivos	La última característica es el año. Debe ser un valor entero positivo (por ejemplo, 2001). Aquí se encuentra de nuevo con un problema en español: los nombres de los atributos no pueden contener la letra "ñ". En este caso se resolvió reemplazar dicha letra y llamar el atributo "anio", que proporciona aproximadamente el mismo sonido. Con las tres características anteriores queda completamente definida una fecha. Esa es la pregunta que debe plantearse en esta etapa: ¿es necesaria más información para describir la entidad que se está representando? Si se encuentra una característica cuyos valores posibles no son simples, como números, cadenas de caracteres, o una lista de valores, hay que preguntarse si dicha característica no es más bien otra entidad que no se identificó en la etapa anterior. Si es el caso, simplemente se debe agregar.



5.3. Relaciones entre entidades



6. Requerimientos no funcionales

El programa se debe poder usar a través de un teléfono celular, o desde un navegador de Internet y el tiempo de respuesta de cualquier consulta ser menor a 0,0001 segundos.



6. ANEXO 2. FORMATO ESPECIFICACIÓN DEL PROBLEMA

Análisis y especificación del problema	
7. Nombre del Sistema:	Darle nombre al sistema
8. Descripción del problema	
Proporcionar la descripción del problema	

9. Identificar los aspectos que hacen parte de un problema	
Cliente	Empresa que requiere el sistema
Usuario	Usuario del sistema
Requerimiento funcional	Identificar y describir los requerimientos funcionales del sistema
Mundo del problema	Identificar todos los elementos del mundo que participan en el problema, con sus principales características y relaciones.
Requerimiento no funcional	Restricciones y condiciones que plantea el cliente, que dependen, en gran medida, del contexto de utilización del programa.

10. Documentación de los requerimientos funcionales del problema		
Definir los requerimientos funcionales o servicios que el programa le ofrecerá al usuario para resolver una parte del problema.		
Nombre		
Resumen		
Entradas		
Resultado		

11. El modelo del mundo del problema	
Describir el mundo en el que ocurre el problema y redactar toda la información necesaria para que el programador pueda escribir el programa.	
Identificar las entidades	
Identificar los elementos del mundo que intervienen en el problema y asociarles un nombre significativo. Dichos elementos pueden ser concretos (una persona, un vehículo), o abstractos (una cuenta bancaria). Una primera pista para localizarlos es buscar los sustantivos del enunciado del problema.	
Entidad	Descripción

11.2. Modelar las características		
A cada característica que ha encontrado, se le debe asociar {1} un nombre significativo, y {2} una descripción del conjunto de valores que dicha característica puede tomar.		
Nombre de la clase:		
Modelado de la clase:		
Atributo	Valores posibles	Comentarios

11.3. Relaciones entre entidades
Identificar las relaciones que existen entre las distintas entidades del mundo y asignarles un nombre. Las relaciones se representan en UML como flechas que unen las cajas de las clases y se denominan usualmente "asociaciones". El diagrama de clases en el cual se incluye la representación de todas las entidades y las relaciones que existen entre ellas se conoce como "el modelo conceptual", porque explica la estructura y las relaciones de los elementos del mundo del problema.

12. Requerimientos no funcionales
Describir las restricciones definidas por el cliente, que dependen, en gran medida, del contexto de utilización del programa. Muchas veces relacionadas con restricciones sobre la tecnología que se debe usar, el volumen de los datos que se debe manejar o la cantidad de usuarios.

13. Prototipo del sistema
Diseñar a modo de prototipo la interfaz o las interfaces que se esperan del sistema atendiendo los requerimientos funcionales del mismo.



REFERENCIAS

- [1] Unicauca. *Conceptos básicos de sistemas de información*. Recuperado de <http://fccea.unicauca.edu.co/old/siconceptosbasicos.htm>
- [2] K. Kendall, J. Kendall, “Administración de Proyectos”, en *Análisis y diseño de sistemas*, K. Kendall, J. Kendall, 8ª edición, Mexico; Pearson, 2011, pp. 57-101.
- [3] J. Villalobos, *Cupi2-Una Solución al Problema de Aprender/Enseñar a Programar*, 2009. Disponible en: http://www.colombiaaprende.edu.co/html/mediateca/1607/articles205832_recurso_1.pdf
- [4] J. Villalobos, R. Casallas, “Comprensión y Especificación de un problema”. *Fundamentos de Programación*, México D.F: Prentice Hall, 2007. pp. 20-60.

